

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	MANDLA VISWATEJA	Reg. No.:	192311399
Section / Batch:	CSE / 2023	Date:	

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big- Θ , Big- Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Code of Conduct

I (J KARTHIK / 192472136) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Backtracking Pseudocode — Q6 to Q10

Q6. Knight's Tour Problem

Input: Board size N (N x N board) **Output:** N x N matrix of move numbers 0..N*N-1 (a valid tour), or "No solution"

```
ALGORITHM KnightsTour(N):
    board[N][N] = -1 for all cells
    board[0][0] = 0
    moveX = [2, 1, -1, -2, -2, -1, 1, 2]
    moveY = [1, 2, 2, 1, -1, -2, -2, -1]
    IF SolveKT(0, 0, 1, board, N) == TRUE:
        PRINT board
    ELSE:
        PRINT "No solution exists"

FUNCTION SolveKT(x, y, moveCount, board, N):
    IF moveCount == N * N:
        RETURN TRUE // all cells visited
    FOR i = 0 TO 7:
        nx = x + moveX[i]; ny = y + moveY[i]
        IF isValid(nx, ny, board, N):
            board[nx][ny] = moveCount
            IF SolveKT(nx, ny, moveCount + 1, board, N):
                RETURN TRUE
            board[nx][ny] = -1 // backtrack
    RETURN FALSE

FUNCTION isValid(x, y, board, N):
    RETURN (0 <= x < N) AND (0 <= y < N) AND board[x][y] == -1
```

Pruning: A move is only taken if it lands on the board and on an unvisited cell (isValid); illegal or already-visited targets are never recursed into.

Complexity: $O(8^{(N^2)})$ worst case before pruning; pruning removes the vast majority of branches in practice.

Q7. Permutation Generation Problem

Input: Array A of n distinct elements **Output:** All n! permutations of A

```
ALGORITHM Permute(A, l, r):
    IF l == r:
        PRINT A // complete permutation
        RETURN
    FOR i = l TO r:
        SWAP(A[l], A[i])
        Permute(A, l + 1, r)
        SWAP(A[l], A[i]) // backtrack, restore order

// Initial call:
Permute(A, 0, n - 1)
```

Pruning: No branch is invalid by definition (every swap yields a valid partial permutation), so the swap-back step is the backtracking action itself; each position is fixed exactly once, avoiding any repeated permutation.

Complexity: Time $O(n * n!)$, Space $O(n)$ recursion depth (in-place swapping, no extra arrays).

Q8. Maze Solving Problem (Rat in a Maze)

Input: maze[N][N] with 1 = open cell, 0 = blocked cell; start (0,0); destination (N-1,N-1) **Output:** solution[N][N] marking a valid path (1 = on path), or "No path exists"

```
ALGORITHM SolveMaze(maze, N):
    solution[N][N] = 0 for all cells
    IF SolveMazeUtil(maze, 0, 0, solution, N):
        PRINT solution
    ELSE:
        PRINT "No path exists"

FUNCTION SolveMazeUtil(maze, x, y, solution, N):
    IF x == N-1 AND y == N-1 AND maze[x][y] == 1:
        solution[x][y] = 1
        RETURN TRUE // destination reached
    IF isSafe(maze, x, y, N):
        solution[x][y] = 1
        IF SolveMazeUtil(maze, x, y+1, solution, N): RETURN TRUE // Right
        IF SolveMazeUtil(maze, x+1, y, solution, N): RETURN TRUE // Down
        IF SolveMazeUtil(maze, x, y-1, solution, N): RETURN TRUE // Left
        IF SolveMazeUtil(maze, x-1, y, solution, N): RETURN TRUE // Up
        solution[x][y] = 0 // backtrack, unmark cell
    RETURN FALSE

FUNCTION isSafe(maze, x, y, N):
    RETURN (0 <= x < N) AND (0 <= y < N) AND maze[x][y] == 1 AND solution[x][y] == 0
```

Pruning: isSafe() rejects out-of-bounds cells, blocked cells (0), and cells already on the current path before any recursive call, so

dead-end and cyclic branches are cut immediately.

Complexity: $O(4^{(N^2)})$ worst case (four directions per cell); pruning via `isSafe()` keeps the explored tree far smaller in practice.

Q9. Combination Sum Problem

Input: `candidates[]` of positive integers (may repeat in a combination), integer `target` **Output:** All unique combinations of candidates that sum exactly to target

```
ALGORITHM CombinationSum(candidates, target):
    SORT(candidates)                // enables early pruning
    result = []
    Backtrack(candidates, target, 0, [], result)
    RETURN result

FUNCTION Backtrack(candidates, remaining, start, path, result):
    IF remaining == 0:
        result.ADD(COPY(path))      // valid combination found
        RETURN
    FOR i = start TO LENGTH(candidates) - 1:
        IF candidates[i] > remaining:
            BREAK                  // sorted -> rest are too large, prune
        path.ADD(candidates[i])
        Backtrack(candidates, remaining - candidates[i], i, path, result)
        path.REMOVE_LAST()          // backtrack
```

Pruning: Sorting first lets the loop BREAK as soon as a candidate exceeds the remaining target; passing `start = i` (not `i+1`) allows reuse while forbidding smaller earlier values, which avoids duplicate combinations without extra checks.

Complexity: Worst case $O(2^{\text{target}})$ branches; sorting costs $O(k \log k)$ for k candidates.

Q10. Word Search Problem (Grid Search)

Input: `grid[N][M]` of characters, target string `word` **Output:** TRUE if word can be traced in the grid via adjacent cells, else FALSE

```
ALGORITHM Exist(grid, word):
    FOR i = 0 TO N-1:
        FOR j = 0 TO M-1:
            IF grid[i][j] == word[0]:
                visited[N][M] = FALSE for all cells
                IF DFS(grid, word, i, j, 0, visited):
                    RETURN TRUE
    RETURN FALSE

FUNCTION DFS(grid, word, x, y, idx, visited):
    IF idx == LENGTH(word):
        RETURN TRUE                // whole word matched
    IF x < 0 OR x >= N OR y < 0 OR y >= M OR visited[x][y] OR grid[x][y] != word[idx]:
        RETURN FALSE              // out of bounds / reused / mismatch
    visited[x][y] = TRUE
    found = DFS(grid, word, x+1, y, idx+1, visited) OR
            DFS(grid, word, x-1, y, idx+1, visited) OR
            DFS(grid, word, x, y+1, idx+1, visited) OR
            DFS(grid, word, x, y-1, idx+1, visited)
    visited[x][y] = FALSE          // backtrack, allow reuse elsewhere
    RETURN found
```

Pruning: The bounds/visited/character check at the top of DFS stops a branch the moment it goes off-grid, revisits a cell, or mismatches the next required letter, so mismatched paths are cut at their very first bad step.

Complexity: $O(N \cdot M \cdot 4^L)$ worst case for word length L ; the character-mismatch check prunes almost all branches immediately.